

Gradient Descent

A training method for Linear Regression

How the model adjusts w_0 and w_1 to reduce Mean Squared Error

Ritvik Genugula Ngoc Mai Nhi Nguyen
Ying Tzu Chen Li Min Yang

MSBA 265 Group Workshop

Linear Regression model and training objective

02

$$\hat{y} = w_0 + w_1x$$

Model

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

Loss

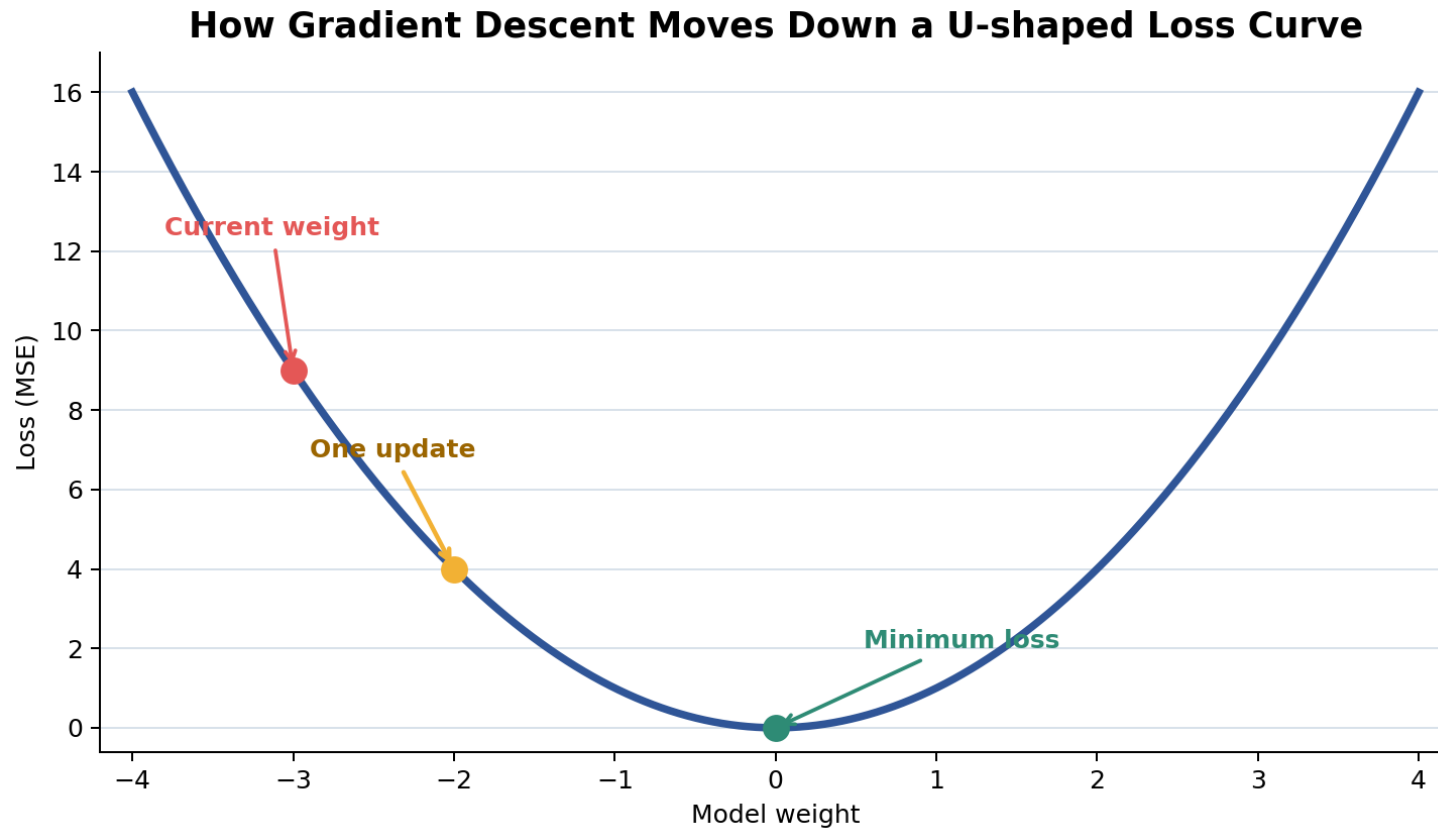
Symbol	Meaning
x	Input feature: BMI
$\hat{y}$	Predicted expenses
w_0	Intercept
w_1	Slope or weight

Training finds values of w_0 and w_1 that make MSE as small as possible.

Lower MSE means smaller squared prediction errors under comparable data and units.

How do we know if we're getting better?

03



Visual element	Meaning
Horizontal position	Possible weight
Height	MSE from that weight
Local slope	Gradient
Bottom	Minimum loss

The point represents a model weight. It does not represent a BMI observation.

How big is the move?

04

$$W_j \leftarrow W_j - \alpha \frac{\partial \text{MSE}}{\partial W_j}$$

Current location	Gradient	Weight movement
Left of minimum	Negative	Moves right
Right of minimum	Positive	Moves left
Near minimum	Near zero	Update becomes small

learning_rate controls the step size

Too small: slow

Balanced: smooth

Too large: overshoots

Workshop value: 0.05

What happens next?

- 1 Predict with the current w_0 and w_1
- 2 Calculate errors for all training observations
- 3 Calculate $\text{grad_}w_0$ and $\text{grad_}w_1$
- 4 Update both weights
- 5 Record MSE at selected checkpoints
- 6 Start the next epoch

$$\frac{\partial \text{MSE}}{\partial w_0} = \frac{2}{n} \sum e_i \quad \frac{\partial \text{MSE}}{\partial w_1} = \frac{2}{n} \sum e_i x_i$$

The full model learns two parameters

Each position on the loss surface represents one pair of w_0 and w_1 values and one MSE value.

Batch means each gradient calculation uses the full training set.

Standardization supports stable training

06

$$X_{\text{std}} = \frac{x - \mu_x}{\sigma_x}$$

Original BMI

Use `x_train` mean and standard deviation

Training scale

Center BMI near 0 and rescale its spread

Original units

Convert the learned coefficients back

$$\text{slope} = \frac{w_1}{\sigma_x} \quad \text{intercept} = w_0 - \frac{w_1 \mu_x}{\sigma_x}$$

Use `x_train` statistics for both train and test data to avoid data leakage.

Python implementation

07

Data

`load_xy()` → Load BMI + expenses

Preparation

`standardize()` → Scale BMI

Training

Gradient Descent loop → Predict → Calculate gradients → Update w_0, w_1 → Repeat

Evaluation

- `mse()` → Measure prediction error
- `r2_score()` → Measure explained variation

Supporting functions

- `print_pipeline_outline()` → Shows workflow
- `print_epoch_learning_rate_explainer()` → Explains training settings
- `print_interpretability_summary()` → Explains final results
- `print_lossBars()` → Displays loss trend

Main workflow

`main()` → Connects everything

Learning rate: 0.05 | Epochs: 10,000 | Full-batch Gradient Descent

Model results

08

$$\text{predicted expenses} = 1852.58 + 369.33 \times \text{BMI}$$

Metric	Train	Test
MSE	139,969,569.72	144,094,972.28
R ²	0.0344	0.0554

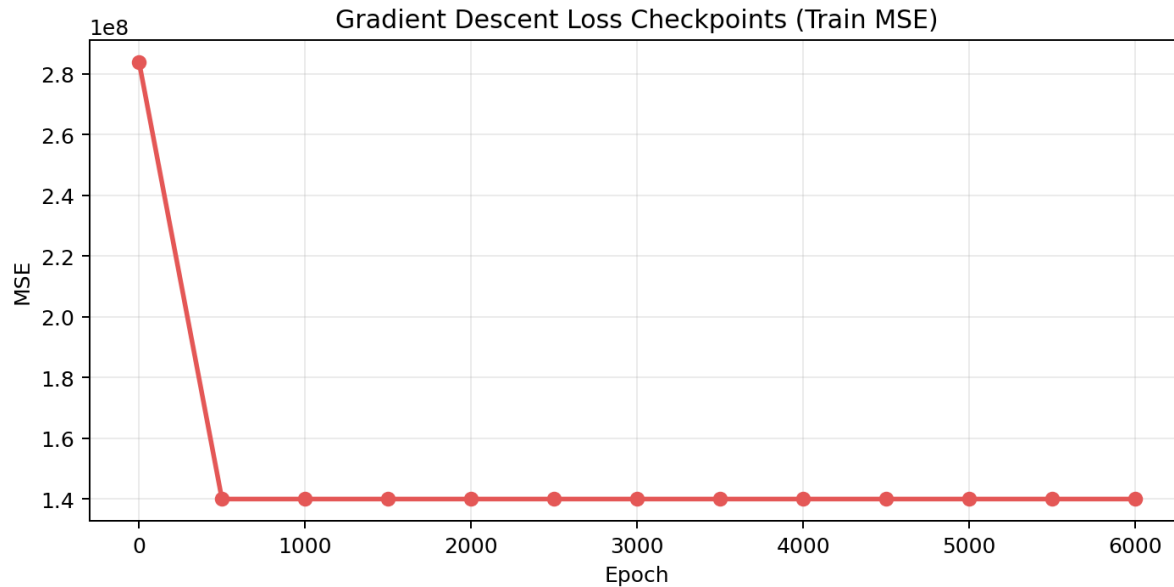
Slope interpretation

Each 1-unit increase in BMI is associated with about 369.34 higher predicted expenses.

Associated with ≠ caused by

Convergence and prediction quality

09



Training loss curve

Sharp drop: the optimizer learns
Flat section: the optimizer converges

Test $R^2 = 0.0554$

Gradient Descent worked

BMI alone predicts weakly

A converged optimizer can still produce a weak model when the selected feature has limited predictive information.

Review

10

- 1 What moves on the U-shaped curve?
- 2 Why does the update subtract the gradient?
- 3 What does learning_rate control?
- 4 What happens during one Batch Gradient Descent epoch?
- 5 Why can loss converge while R^2 remains low?

Gradient Descent trains Linear Regression by adjusting w_0 and w_1 in the direction that reduces MSE.